

On the Use of Artificial Intelligence in Software Testing: State of the Art and Application to Satellite Systems

ALEJANDRO J. CALDERÓN, Ikerlan Technology Research Centre, Spain

RICARDO ARNAIZ, Ikerlan Technology Research Centre, Spain

JANNIS WOLF, Barcelona Supercomputing Center (BSC), Spain

MAR HERNÁNDEZ, Ikerlan Technology Research Centre, Spain

STEFANO SINISI, Collins Aerospace, Italy

VALERIO DI VALERIO, Collins Aerospace, Italy

GIULIA STAZI, Collins Aerospace, Italy

LEONIDAS KOSMIDIS, Barcelona Supercomputing Center (BSC), Spain

The integration of Artificial Intelligence (AI) into software testing has emerged as a promising approach to addressing the limitations of conventional methods, particularly in safety-critical domains such as automotive and aerospace. Traditional software testing is often manual, resource-intensive, and prone to incomplete coverage. In contrast, AI-based techniques have been applied to automate test case generation, optimise testing processes, and improve defect prediction. This study extends these approaches to the context of Embodied AI, where the reliability of systems that perceive and act within complex environments is a primary concern. Autonomous satellites are considered as representative embodied agents, for which the robustness of the perception-action loop is mission-critical.

The paper presents two main contributions. First, it reviews the state of the art in the application of AI techniques to software testing. Second, it demonstrates the practical viability of these techniques through a proof of concept (PoC) in the satellite domain. In this PoC, generative AI is used to create synthetic datasets simulating adverse atmospheric conditions, including varying levels of cloud opacity. These datasets are used to evaluate and improve an on-board object detection model. Experimental results indicate that this approach reveals limitations of the baseline model and, following retraining, enhances robustness under challenging conditions while maintaining performance in clear scenarios. The findings demonstrate the potential of AI-based testing methodologies to support the validation and improvement of mission-critical embodied systems.

Additional Key Words and Phrases: Artificial Intelligence, Software Testing, Satellite Systems

1 Introduction

The rapid development of AI has enabled advances in automation, optimisation, and decision-making across multiple domains. A critical area of application is its integration into embodied agents, which are physical systems such as robots, autonomous vehicles, and satellites that perceive, process information, and act within the

Authors' Contact Information: Alejandro J. Calderón, Ikerlan Technology Research Centre, Mondragon, Spain, ajcalderon@ikerlan.es; Ricardo Arnaiz, Ikerlan Technology Research Centre, Mondragon, Spain, rarnaiz@ikerlan.es; Jannis Wolf, Barcelona Supercomputing Center (BSC), Barcelona, Spain, jannis.wolf@bsc.es; Mar Hernández, Ikerlan Technology Research Centre, Mondragon, Spain, mar.hernandez@ikerlan.es; Stefano Sinisi, Collins Aerospace, Rome, Italy, stefano.sinisi@collins.com; Valerio Di Valerio, Collins Aerospace, Rome, Italy, valerio.divalerio@collins.com; Giulia Stazi, Collins Aerospace, Rome, Italy, giulia.stazi@collins.com; Leonidas Kosmidis, Barcelona Supercomputing Center (BSC), Barcelona, Spain, leonidas.kosmidis@bsc.es.

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.

© 2026 Copyright held by the owner/author(s). Publication rights licensed to ACM.

ACM 2378-9638/2026/2-ART

<https://doi.org/10.1145/3783988>

physical world. This field, referred to as Embodied AI, extends intelligence beyond digital domains into dynamic environments where failures can have significant consequences. Satellites, for example, operate as autonomous embodied agents that rely on sensors to perceive their surroundings and perform mission-critical tasks without continuous human supervision. Ensuring the reliability of their software is therefore critical.

Traditional software testing faces substantial challenges when applied to Embodied AI. Conventional methods are resource-intensive and limited in their ability to account for the diversity and unpredictability of real-world scenarios [15]. They are often unable to capture the continuous and uncertain nature of environmental inputs, leaving gaps in coverage that may allow critical errors to persist. In the case of a satellite tasked with maritime surveillance, such limitations could result in the failure to detect vessels under adverse weather conditions, which would represent a shortcoming of the perceptual system with direct operational consequences.

AI-driven testing techniques offer an approach to address these challenges [27]. Methods such as Machine Learning (ML), Deep Learning (DL), and Generative Adversarial Networks (GANs) are increasingly being used to automate test case generation, generate synthetic input data, and optimise the selection and prioritisation of test scenarios [7]. By complementing conventional testing, these approaches enable more efficient, scalable, and systematic validation of embodied agents in complex environments.

This paper investigates the role of AI in software testing through two main contributions. First, it presents a thorough review of the state of the art of AI applications in software testing, organised into a taxonomy of three categories: automated generation of test cases and synthetic data, intelligent prioritisation of test cases, and anomaly detection and defect prediction. Second, it demonstrates a proof of concept in the context of the Horizon Europe METASAT project [33], applying AI-driven testing to the perceptual software of a satellite system. In this case study, generative AI is used to produce synthetic aerial images with varying levels of cloud cover, which enables systematic evaluation and improvement of a ship detection model under conditions that are otherwise difficult to reproduce. The results illustrate how AI-driven testing can enhance the robustness of mission-critical embodied systems by identifying weaknesses and supporting retraining under adverse conditions.

The research is motivated by the growing importance of AI in improving the accuracy, efficiency, and reliability of the software testing process. Although many individual AI techniques have been developed and applied in this field, a comprehensive understanding of how they can be systematically integrated and optimised is still evolving. This paper aims to contribute to that understanding by providing a detailed review of AI applications in this field, offering insights into their potential benefits and limitations, and suggesting ways to apply these techniques in real-world scenarios.

The remainder of this paper is structured as follows. Section 2 reviews the state of the art and introduces the proposed taxonomy. Section 3 describes the methodology used to apply the selected AI techniques to the satellite-related proof of concept. Section 4 presents the evaluation and results and discusses the impact of the applied AI techniques. Finally, Section 5 concludes the paper by summarising the findings.

2 State of the art

Testing is a critical phase in the development cycle of any software/hardware system, ensuring that the final product meets its functional, performance, and security requirements. Traditionally, testing has been a time-consuming and resource-intensive process, often involving manual test case creation, execution, and defect detection. As systems grow in complexity, particularly in mission-critical domains such as aerospace, automotive, and telecommunications, these traditional approaches to testing struggle to keep up. To address these challenges, AI has emerged as a powerful tool that can automate and enhance various aspects of the testing process.

AI techniques have revolutionised testing by automating test case generation, optimising test coverage, predicting defects, and even automating test execution. ML models, GAs, and DL methods are increasingly being used to identify defects early, improve test efficiency, and reduce testing time. By applying AI, testing becomes

not only faster but also more comprehensive, allowing for better detection of edge cases and bugs that might otherwise go unnoticed. Furthermore, AI-driven approaches enable adaptive testing, where tests evolve based on real-time feedback, ensuring that testing remains aligned with changes and new development requirements.

This section provides an overview of the state of the art in AI applications for software testing. Through an analysis of recent research, we observed that the majority of AI-driven testing efforts align with three dominant categories, which we use to structure this review: (1) automated generation of test cases and synthetic data, where AI algorithms create diverse testing scenarios and complementary testing data; (2) intelligent prioritisation of test cases, which focusses on dynamically selecting high impact test sequences to maximise coverage and efficiency; and (3) anomaly detection and defect prediction, leveraging AI to identify irregularities in system behaviour, predict failure-prone components, and minimise human oversight in debugging. These categories emerged from a systematic examination of recent literature as they collectively encapsulate the most prevalent and impactful advances in AI-based testing.

The review examines how these approaches address current challenges in testing processes. For each category, we discuss specific AI methods, emphasising their benefits, limitations, and areas of ongoing research. By organising the state of the art in this way, we highlight both the maturity of existing solutions and the critical gaps that future work must address.

2.1 Automated generation of test cases and synthetic data

Automated generation of test cases and synthetic data is essential for modern software testing, as it addresses challenges such as low test diversity, inadequate coverage, and high manual effort. Recent advances in AI and ML have enabled novel approaches to address these challenges, ranging from generative models to optimisation and Natural Language Processing (NLP) techniques. This section reviews state of the art methods for generation of test cases and synthetic data, categorised by their core approaches.

Generative Adversarial Networks (GANs) have been widely adopted to produce synthetic data and diverse test cases. Guo et al. [24] presented a GAN-based framework that increases test diversity by generating realistic test cases to improve test coverage. Expanding on this, Guo et al. [25] introduced a GAN-based approach for the generation of test data, using execution path information to train a generative model. Their method applied Wasserstein GAN with Gradient Penalty (WGAN-GP) to achieve high branch coverage without requiring detailed branch analysis, demonstrating effectiveness in large-scale testing scenarios.

Sun et al. [65] enhanced GANs by integrating a Variational Autoencoder (VAE), creating a VAE-GAN model that stabilises training and increases the variety of test cases, improves code coverage, and discovers unique vulnerabilities of the system. Similarly, Qin et al. [50] proposed a Conditional WGAN-GP (CWGAN-GP) model that generates test cases conditioned on specific branches, which expanded coverage and exposed additional vulnerabilities across various programs.

In applications beyond direct code testing, Dewi et al. [19] used Deep Convolutional GANs (DCGANs) to produce synthetic traffic sign images, which improved performance in traffic sign recognition systems. Although focused on image recognition, this study illustrated how combining synthetic and real data can enhance detection, a strategy that is also applicable to software test cases generation.

Evolutionary Algorithms (EAs), especially GAs, are prominent in automated test case generation. Rajagopal et al. [53] applied adaptive GAs to optimise path-focused test cases, dynamically adjusting crossover and mutation rates to escape local minima. Gang et al. [21] examined various genetic and swarm-based algorithms, demonstrating that these methods effectively cover a wide input domain and reveal hidden faults, ultimately improving the quality of the software.

Rani et al. [54] addressed regression testing challenges by applying GA to automate the generation and prioritisation of test cases. Their approach minimised redundancy and optimised testing for changed areas,

reducing the overall cost and time of regression testing. Arasteh et al. [9] further improved traditional GAs by hybridising with Grey Wolf Optimisation to achieve high branch coverage, while Sasikumar et al. [58] implemented a Hybrid Genetic Algorithm (HGA) to maximise mutation scores and path coverage. These hybrid approaches showed improved efficiency and coverage in benchmark tests.

Al-Masri et al. [4] used a teaching-learning-based optimisation algorithm to reduce redundancy while ensuring complete path coverage, achieving complete coverage in Java-based programs. Olsthoorn et al. [47] suggested a multi-technique approach, combining grammar-based fuzzing, ML, and EAs, hypothesising that such integration would improve the structure and pattern preservation of test cases.

Model-Based Testing (MBT) approaches use models like Unified Modelling Language (UML) to automate the generation of test cases. Naimi et al. [42] presented a framework to translate UML diagrams into test cases using generative AI, simplifying the generation of test cases and improving early-stage coverage. Naz et al. [43] used UML sequence and class diagrams to generate test cases through a novel EA named U-NSGA-III, achieving superior statement coverage and faster convergence compared to traditional GAs and Particle Swarm Optimisations (PSO).

In a similar approach, Worku et al. [69] developed an ML-based model to generate test cases focused on quality attributes, ensuring the generation of relevant test cases that address crucial quality concerns. This approach, leveraging ML algorithms like Support Vector Machine (SVM) and Random Forest (RF), optimises test generation for quality-critical applications.

NLP techniques facilitate test case generation by interpreting software documentation and requirements. Tufano et al. [67] used a transformer model to automatically generate assertion statements for unit tests, enhancing test cases for higher coverage. Their approach achieved significant improvements over previous models, making unit test creation more accessible to developers.

Mathur et al. [37] applied transformer models such as T5 and GPT-3 to generate context-based test cases, using conversation data to cover complex requirements comprehensively. By extracting relevant keywords and context, their approach reduced the reliance on human expertise and expanded scenario coverage, especially for complex systems.

Enhanced fuzzing techniques integrate ML models to improve input diversity and vulnerability detection. Nie et al. [44] used a Long Short-Term Memory (LSTM) GAN for protocol fuzzing, achieving greater vulnerability detection than conventional tools such as AFL and Peach. Qin et al. [50] introduced CWGAN-GP for conditional test generation based on branch coverage, enhancing fuzzing performance, and expanding the exploration of vulnerabilities in security-critical software.

ML and data mining techniques provide adaptive test generation by learning from historical testing data. Khudhur et al. [32] used K-Nearest Neighbours (KNN) to detect patterns in previous test cases, prioritising fault-prone areas for testing. This approach enabled dynamic test case prioritisation, adapting to changing application features.

Liu et al. [35] combined GAs with a Radial Basis Function (RBF) neural network, optimising the fitness values of the test case for branch coverage. The proposed model (GAR) outperformed traditional GAs by reducing iteration counts and improving branch coverage, showing promise for ML-enhanced test generation.

2.2 Intelligent prioritisation of test cases

Test Case Prioritisation (TCP) is crucial for optimising regression testing, especially in dynamic fields such as Continuous Integration (CI). TCP seeks to reorder test cases to maximise fault detection early in testing, enhancing efficiency, and minimising costs. Recent advances in ML, Reinforcement Learning (RL), and unsupervised approaches have expanded the capabilities of TCP. This section reviews the latest approaches in intelligent test case prioritisation, organised by their methodological focus.

RL has shown promise in TCP, particularly in CI environments where adaptability to constant code changes is essential. Bagherzadeh et al. [11] explored RL in CI by framing TCP as a ranking problem, where the RL agent continuously learns an optimal prioritisation strategy through sequential interactions. This approach demonstrated significant accuracy improvements, bringing prioritisation closer to optimal. Similarly, Chen et al. [16] extended the RETECS framework by developing a prioritisation method for new test cases lacking historical data, enhancing fault detection through an additional reward structure.

Other RL-based studies addressed the challenge of TCP of responding rapidly to new changes in CI. For example, Mirzaei et al. [39] investigated reward functions, categorising them into current and historical information-based functions, with their Historical Failure Count With Time Window (HFCW) reward function outperforming others in fault detection. Omri et al. [48] employed an online learn-to-rank RL model, prioritising test cases based on their likelihood of failure while executing only 40% of available test cases, which still detected over 95% of failures. Da Roza et al. [18] applied a sliding window method for TCP, instantiating it with RF and LSTM models; RF was particularly effective in time-constrained budgets.

Several studies focused on supervised ML models to improve TCP by leveraging historical test data and features related to CI contexts. Abdelkarim et al. [1] introduced TCP-Net, a Deep Neural Network (DNN) tool for TCP that correlates the characteristics of the source code, the metadata of the test case and the failure history to generate prioritised test cases. Their experimental results on industrial data sets demonstrated the capability of TCP-Net to effectively learn complex relationships between code changes and faults.

Zhao et al. [73] conducted a large-scale study of ML-based TCP techniques, analysing 11 algorithms across 11 open-source subjects to assess performance stability. They found that pre-training with cross-subject data, followed by fine-tuning, significantly improved performance, with the pre-trained MART model producing the optimal sequence in 80% of the cases. Yaraghi et al. [70] built on these findings by proposing a comprehensive feature set and data model tailored to CI environments, incorporating various data sources for TCP, which proved to be effective for extended regression tests.

Sawant et al. [59] presented a ML-based ensemble TCP approach, detailing practical considerations for integrating TCP into CI/Continuous Deployment (CD) pipelines. Their study provided insights on data requirements and model selection strategies, demonstrating the viability of advanced ML for real-world regression testing. Huang et al. [29] proposed a learn-to-rank model using RF, which outperformed single heuristic approaches, increasing the fault detection rate in model-based regression testing scenarios.

Hybrid approaches combining multiple algorithms have emerged to tackle TCP's specific challenges in CI environments. For example, Kaur et al. [31] utilised the Harris Hawks Optimisation (HHO) algorithm, inspired by nature, to select efficient test cases in regression testing. This method draws on the collaborative hunting behaviour of hawks, selecting an optimal subset of test cases through diverse directional strategies, which is particularly suitable for reducing the size of the test suite without compromising effectiveness.

Guaceanu et al. [20] proposed RoughTCP, an unsupervised approach utilising rough set-based clustering to handle the high variance in CI data. By clustering test cases based on intrinsic patterns like faults and run duration, RoughTCP outperformed supervised methods on high budgets and even outperformed traditional methods on lower budgets, highlighting the potential of unsupervised learning in dynamic environments. Zafar et al. [71] optimised MBT test suites using ML, similarity measures, and greedy algorithms, achieving a test suite reduction of up to 92% while maintaining fault detection rates between 95% and 100%.

In practice, the selection of TCP techniques can depend heavily on the constraints of the CI/CD environment and the trade-offs between time, cost, and fault detection rates. Rosenbauer et al. [57] compared RL-based TCP methods, revealing that the XCS classifier system produced more stable results than neural networks, making it suitable for industrial applications. Waqar et al. [68] emphasised the importance of test suite minimisation in regression testing and presented a RL approach evaluated in multiple Android applications, which consistently outperformed baseline approaches.

Huang et al. [28] developed a Generative Adversarial Imitation Learning (GAIL) approach for TCP, allowing agents to learn from expert rankings without relying on potentially biased reward functions. The study introduced a new metric, APFDET, specifically designed for CI environments, demonstrating the superior fault detection capability of GAIL in industrial data sets. This focus on comparative metrics underscores the increasing sophistication and adaptability of intelligent TCP techniques.

The research on intelligent TCP highlights the shift toward adaptive, data-driven techniques that can meet the needs of CI environments. RL and ML models, whether individually applied or integrated into hybrid systems, show significant improvements in prioritisation accuracy and fault detection. Unsupervised clustering approaches, such as RoughTCP, indicate promising results in environments where labeled data is limited. In addition, hybrid approaches, such as those that combine RF with sliding-window methods or ensemble methods, provide flexible solutions that adapt to evolving testing demands.

2.3 Anomaly detection and defect prediction

Anomaly detection and defect prediction play crucial roles in improving software quality by identifying areas prone to defects early in the development lifecycle. Leveraging ML and DL techniques has improved defect prediction accuracy and reduced testing costs. This review discusses recent approaches in anomaly detection and defect prediction, categorised by their core methodologies and contributions.

Traditional ML techniques are still widely used for defect prediction, and many studies focus on optimising classifiers to improve predictive performance. Ali et al. [6] presented a classification framework based on ensemble learning and feature selection, showing that their ensemble-based model outperformed ten conventional classifiers, including Naive Bayes (NB), SVM, and Decision Trees (DT), on NASA datasets. Similarly, Assim et al. [10] explored various ML algorithms, including Artificial Neural Networks (ANNs), RF, and Gaussian Processes (GP), identifying SMOreg as the best performer.

Mehmood et al. [38] demonstrated that feature selection improves the accuracy of defect prediction by applying this approach to multiple NASA datasets. Their study showed that classifiers such as RFs and SVMs benefited significantly from feature selection, highlighting its impact on classification performance. Gurram et al. [26] used a genetic neural network model to address dataset imbalance issues, utilising GAs to fine-tune the network's features and hyperparameters, achieving improved accuracy in detecting defect-prone code areas.

DL techniques, especially Convolutional Neural Networks (CNNs), have gained traction for defect prediction due to their ability to automatically learn complex data patterns. Nirmalraj et al. [45] demonstrated improved accuracy over Region-based CNN (RCNN) and Fast-RCNN (F-RCNN) proposing a change-level defect prediction model using Scale-Aware F-RCNN (S-AF-RCNN), highlighting the potential of DL for just-in-time (JIT) defect prediction. Similarly, Tang et al. [66] used LSTM networks to predict memory leaks in software, showing that LSTMs are well suited for capturing sequential dependencies in code, particularly for identifying resource depletion issues.

Shankar et al. [61] integrated supervised and unsupervised learning techniques for defect prediction, using feature selection methods such as Recurrent Feature Elimination (RFE) and SelectKBest to improve classifier performance. Their comparative analysis across models including KNN, CNN, and Gaussian Naive Bayes (GNB), identified ensemble learning as highly effective. Abdu et al. [2] introduced a hierarchical CNN model that combined syntax and semantic graph features to improve defect prediction accuracy, demonstrating that multi-representation models outperform traditional single-representation approaches.

Ensemble techniques and Automated Machine Learning (AutoML) approaches have been effective in optimising defect prediction performance by combining multiple models. Basgalupp et al. [13] adapted an AutoML approach called HEAD-DT to software defect prediction, which automatically selects and tunes decision tree models. HEAD-DT outperformed traditional ML models, underlining the potential of AutoML in defect prediction.

Ali et al. [5] developed a two-stage ensemble model combining RF, SVM, NB, and ANN, which achieved high precision in NASA datasets. Their approach involved iterative parameter optimisation, with the final predictions determined by a voting ensemble that outperformed numerous state-of-the-art methods. Aquil et al. [8] confirmed the robustness of ensemble methods for defect prediction, with their study on 13 datasets showing ensemble techniques as consistently high performers.

Neural networks, particularly CNNs and Recurrent Neural Networks (RNNs), are increasingly applied to defect prediction due to their adaptability and success in various domains. Qiu et al. [51] leveraged CNNs to learn feature representations from software datasets, achieving better predictive accuracy than traditional methods. Their approach emphasises the potential of CNNs in identifying defect patterns in complex data. Borandag et al. [14] demonstrated the efficacy of RNNs over traditional ML classifiers on larger datasets, with RNNs showing a significant improvement in predictive accuracy.

Rifat et al. [56] combined ANNs with RFs, achieving high precision on defect-prone data. Their study highlighted the synergy between ML classifiers and ANNs, with RF-ANN combinations exceeding individual classifier performance.

JIT defect prediction aims to identify defects at the change level, prompting immediate feedback during development. Luo et al. [36] proposed a JIT defect prediction model using GANs to handle semantically preserved code changes, a common issue in traditional JIT models. Their GAN-based approach detected defects in semantic equivalent code, demonstrating improved performance over conventional JIT models.

Incorporating semantic features has proven to be effective in capturing complex code structures. Moshtari et al. [41] introduced an unsupervised approach using KNN-based outlier detection to identify defect-prone components without labelled data, achieving high precision in multiple projects by focussing on outliers.

Cross-project defect prediction enables models trained on one project to predict defects in another, a challenge due to data variations. Setiawan et al. [60] employed neural networks in NASA archival data, showing that neural organisation techniques improve prediction accuracy in diverse datasets. Srinivasarao et al. [62] evaluated ML models such as RNNs for defect prediction, highlighting the advantages of RNNs in learning sequential patterns applicable to cross-project prediction scenarios.

Comparative analyses reveal the evolving landscape of defect prediction methodologies. Panda et al. [49] discussed the Synthetic Minority Oversampling Technique Edited Nearest Neighbour (SMOTE ENN) method to handle imbalanced data sets, a common issue in defect prediction. Their study found that RF models, optimised for class imbalance, achieved notable fault prediction accuracy. This focus on class balancing aligns with the increasing adoption of pre-processing techniques in defect prediction.

Research on anomaly detection and defect prediction reflects a shift toward hybrid models, ensemble techniques, and DL-based solutions that capture complex defect patterns. Cross-project learning and JIT defect prediction represent growing areas, with neural networks and GANs enhancing defect prediction by learning semantic features. Unsupervised and AutoML approaches also show potential for scalable solutions, reducing dependency on labelled data and manual parameter tuning. Together, these advancements signify progress toward highly adaptable, accurate, and resource-efficient defect prediction models.

To summarise the discussion presented in this section, Figure 1 provides a taxonomy of AI-driven software testing. The figure organises the state of the art into the three main categories: automated generation of test cases and synthetic data, intelligent prioritisation of test cases, and anomaly detection and defect prediction, each including a range of representative approaches. This taxonomy offers a structured perspective that facilitates the identification and understanding of current approaches and their objectives.

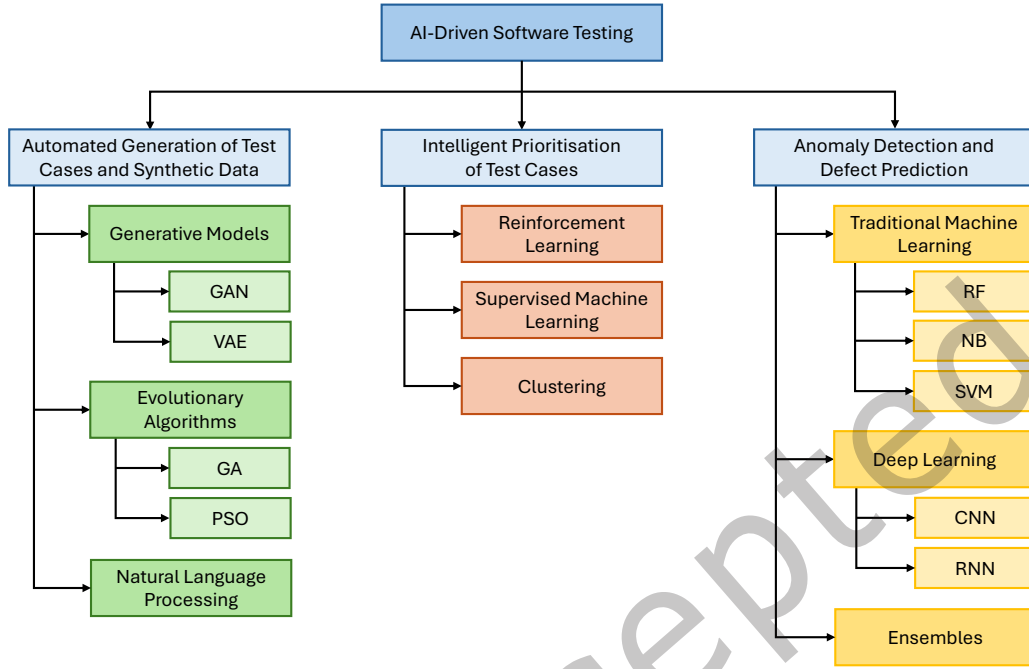


Fig. 1. Taxonomy of AI techniques used for software testing

3 Application to satellite systems: A case study in Embodied AI

The validation of an Embodied AI system requires demonstrating its reliability within the operational environment. A satellite can be considered a representative embodied agent, as it is a physical system that operates autonomously in the complex and unpredictable environment of space. Its on-board software is responsible for perception, reasoning, and action, making its validation not only a matter of code verification but also a central challenge in Embodied AI research. This section presents a practical case study that illustrates how AI-driven testing techniques, specifically the generative methods surveyed in Section 2.1, can be applied to validate the perceptual system of such an agent.

Our case study addresses a major difficulty in developing software for embodied agents: the limited availability of diverse, real-world operational data. Acquiring comprehensive datasets for training and testing is often prohibitively expensive or logistically complex, and reproducing rare but critical scenarios may be infeasible. Generative AI techniques provide a way to mitigate this data gap. To demonstrate this, we present a PoC developed within the Horizon Europe METASAT project [33]. The use case involves a satellite application for maritime surveillance, where the system detects ships in aerial images and marks them with bounding boxes. This perception task is fundamental for an autonomous surveillance agent. The application is part of OBPMML, which belongs to the open-source OBPMML (On-Board Processing Benchmarks) benchmarking suite [63][64] of the European Space Agency (ESA). The application uses a neural network architecture based on a YOLOX model [22] to detect and annotate ships in images with aligned bounding boxes. The dataset used to train this neural network is the open data from the Airbus Ship Detection Kaggle challenge [3], which contains more than 200000

images. However, only 7000 of these images were randomly selected to build the training (5000), validation (1000), and testing (1000) datasets. An example of how the application works can be seen in Figure 2, where the ship is surrounded by a red bounding box, showing that the application has detected it correctly.

3.1 Objective: Testing the robustness of an embodied perception system

The primary objective of this PoC is to demonstrate how generative AI can be used to test and enhance the robustness of the perception system of an embodied agent. Performance under ideal conditions, such as the clear, cloud-free images that dominate the training set, does not adequately reflect real-world reliability. A reliable system must also maintain effectiveness under the adverse and varied conditions it is expected to encounter.

In this case study, we consider cloud cover, a common environmental factor for Earth-observation satellites. Clouds may partially or fully obscure objects, creating a significant challenge for detection models. The objective of our study is twofold: (1) to quantitatively assess the degradation of detection performance as cloud opacity increases, and (2) to demonstrate that augmenting training data with synthetically generated adverse conditions can improve model robustness without requiring the collection of new real-world data.

To address this objective, we evaluate two distinct yet complementary approaches for generating synthetic test data, which are described in the following subsections.



Fig. 2. An example of how the detection application works in a real image

3.2 Approach 1: Generation of new images

The first approach involves using neural networks to generate completely new synthetic images, either through pre-trained models or newly developed ones. The goal of this approach is to generate images similar to those of the original dataset used to train the detection application, allowing the ship detection tool to be evaluated on a wider range of images with greater diversity. Two different strategies have been explored within this approach: training a new model and using available pre-trained models.

For training, two state-of-the-art neural networks were evaluated. Both architectures are based on the original GAN method presented by Goodfellow et al. [23] back in 2014. One of the two trained models was based on a DCGAN architecture [52]. DCGAN is a combination of GANs and CNNs that is used to produce very realistic images. The second model tested was based on StyleGAN [30], a more advanced architecture that supports a

larger image size while ensuring more stable training. This is achieved by iterating through different image sizes, starting with a very small image size (e.g., 8×8 pixels) and progressively doubling the size until the desired resolution is reached.

When using pre-trained models, the use of text-to-image models and 3D model to 2D image frameworks were evaluated. Using these pre-trained models, developers can employ prompts and 3D models to investigate corner cases that may reveal vulnerabilities in the detection application, such as partially obscured ships or vessels located inland. Among text-to-image models, one generative model that has gained significant attention in recent years is the diffusion model and its different variants. With respect to 3D models to 2D images, there exist several frameworks. These frameworks typically begin with a basic 3D model of the scene, such as objects (e.g., ships), and the environment (e.g., open sea, ports). By adjusting various parameters, such as the placement of objects and their interactions with the environment, these frameworks can generate thousands of different images from the initial 3D model.

3.3 Approach 2: Modification of existing datasets

In contrast to the first one, this approach focusses on modifying existing images by adding different conditions or noise to the images, simulating scenarios that would otherwise require a lot of time to obtain in real life. The goal is to create a large and diverse set of images to thoroughly evaluate the performance of the detection model. However, the path to achieve this expansion of the dataset is completely different. Instead of generating entirely new images, the focus is on modifying and augmenting existing ones to increase the diversity of the dataset. Several classic data-augmentation techniques can be used to achieve this, such as adding noise blur or different lighting. In addition, by leveraging AI image generation capabilities, it is possible to model different weather effects, in this case clouds obstructing the view. This approach is particularly valuable, as obtaining such images and ground-truth labels in the real world could take a lot of time or even be impossible depending on the target application. Therefore, the ability to generate these conditions synthetically presents a promising and efficient alternative.

The goal of this approach is to demonstrate that, by using synthetically generated images, the performance of the model can be evaluated under conditions different from those in the training set. In addition, by retraining the model with these synthetic images, its accuracy can be improved, allowing it to work better under broader conditions. AI models, regardless of their application and how they are trained, can only learn from the data to which they are exposed during training. In this case, the training subset predominantly consists of cloud-free images, making it challenging to accurately assess the performance of the model under cloudy conditions. Given that, incorporating clouds into the dataset presents a highly relevant application since it is a scenario the detection model would certainly encounter, and having the opportunity of evaluating and improving its performance in such situations is crucial. To generate clouds in the original dataset, the Satellite Cloud Generator tool [17] has been selected. The proposed framework combines several novel techniques for cloud simulation, including cloud locality degree, ground shadow, and automated segmentation masks. In addition, it integrates established methods from the literature, such as cloud colour, channel misalignment, and ground blurring, within a unified approach.

To validate this PoC, the Mean Average Precision (mAP) metric is first calculated for the detection model on both the original dataset and the various synthetic datasets generated. Subsequently, the model is retrained using different synthetic datasets and the mAP is recalculated again across all datasets.

4 Evaluation of the selected AI techniques

This section evaluates the effectiveness of the generative AI techniques described in the previous section for testing and enhancing the robustness of the perception system within our embodied agent. We begin by assessing

the performance of the initial model under synthetically generated adverse conditions. We then demonstrate the improvements achieved after retraining the model with the generated synthetic data. Together, these results provide empirical evidence supporting the value of AI-driven testing in validating and strengthening the reliability of Embodied AI systems.

4.1 Generation of new images

As explained in Section 3, the evaluation for the generation of new images is divided into two sections, distinguishing between training new models or using pre-trained ones.

4.1.1 Training GAN models. To evaluate the feasibility of training a generative model from scratch for our testing scenario, we selected two representative GAN architectures for comparison: DCGAN and StyleGAN. This choice was not intended as an exhaustive survey of all GAN variants, but rather as a targeted comparison of two distinct design approaches. DCGAN was chosen as a baseline. As one of the earliest GAN architectures to achieve stable training for image generation, it is well understood, comparatively simple, and computationally efficient. Its evaluation provides a performance baseline and illustrates the capabilities of a classic GAN architecture on our dataset. StyleGAN was selected to represent a more advanced architecture. It is known for producing high-resolution, photorealistic images and for achieving stable training at larger image sizes. Its evaluation allows us to assess the potential benefits of a state of the art, but more computationally demanding, model. By comparing these two models, we aim to analyse the trade-offs between architectural complexity, computational resource requirements (training time and memory), and the quality and suitability of generated images for software testing purposes.

The dataset used to train both models was the same as the one used in the application detection model: the open dataset from the Airbus Ship Detection Kaggle challenge. However, to reduce training time and ensure a balanced set, the initial dataset was preprocessed by discarding images with very low file size and performing random undersampling, reducing the total number of images to 45000.

The first model trained was based on DCGAN. GAN models consist of two components: a generator and a discriminator. The generator synthesises the images, while the discriminator evaluates their authenticity by distinguishing between real and generated samples. Through an adversarial training process, the generator progressively improves its ability to produce realistic images. Once the discriminator can no longer reliably differentiate between real and generated images, the GAN achieves the capability to generate highly realistic outputs. The generator is composed of six transposed convolution layers, each of them paired with batch normalisation and ReLU activations, except for the last layer that has a Tanh activation. On the other hand, the discriminator is composed of six convolutional layers, with batch normalisation and LeakyReLU activations in the inner layers, while the last layer has a sigmoid activation. After 100 epochs of training, with a batch size of 128 images, the results of training can be seen in Figure 3. The generated synthetic images began to resemble the original dataset. However, there were two main problems. The first was the diversity of the output. The original dataset contains a wide range of images, including very small ships and, in some cases, a significant amount of land. Despite the number of training epochs, the model struggled to focus on the ships. The second problem was the image size. Most DCGAN implementations use 128×128 pixels as the image size. Beyond that size, the architecture is not optimised, leading to unstable training. Given that the input size for the detection application is 768×768 , this model architecture was not suitable.

The second model tested was based on StyleGAN, which supports a larger image size while ensuring more stable training. In addition to this, further preprocessing was applied to reduce the diversity of the dataset. Only images featuring large ships were selected to ensure that the model focused more on the ships than on the surrounding environment. After this, the number of images was further reduced to 8500 images.

Fig. 3. DCGAN training results with image size 128×128 pixels

After training the model for 30 epochs with a batch size of 128 in smaller image sizes and 120 epochs with a batch size of 4 for 512×512 pixels image size, the training results can be seen in Figure 4a. The results started to look like a ship, and, even with this low-resolution image, the detection model was able to detect a ship in the image, as shown in Figure 4b. It should be noted that the output image size of the model was 512×512 pixels, so it had to be rescaled to 768×768 pixels.



(a) StyleGAN training result

(b) Synthetic image detected by the application

Fig. 4. Synthetic images rescaled to 768×768 pixels.

This model proved to be more promising than the initial one; however, it also had its limitations. The primary challenge was that the memory and time required to train the model grew exponentially with the image size. The Graphics Processing Unit (GPU) used for the training was an NVIDIA RTX 4090 with 24 GB of memory. Using this device and the epochs mentioned above, it took more than three days to finish the training, during which the memory was fully used, to obtain these results. Given the computational demands of training a new model to produce high-quality images, we decided to explore the use of pre-trained models.

4.1.2 Using pre-trained models. When using pre-trained models, the use of text-to-image models has gained a lot of attention in recent years, especially the diffusion model and its different variants [40]. An image generated with a Stable Diffusion XL model from stability.ai [46] and its bounding box can be seen in Figure 5. These models accept not only positive prompts (what they should generate), but also negative prompts (what they should not generate), as well as different parameters, such as the number of steps in the diffusion process. Given that, it is possible to obtain a lot of very realistic images with great quality, variety, and resolution with only a few adjustments to these inputs. Apart from this, it is possible to ask for corner cases, where maybe the ship is inland, and check the performance of the model in these difficult scenarios. Overall, the use of text-to-image models for testing is a very promising solution.

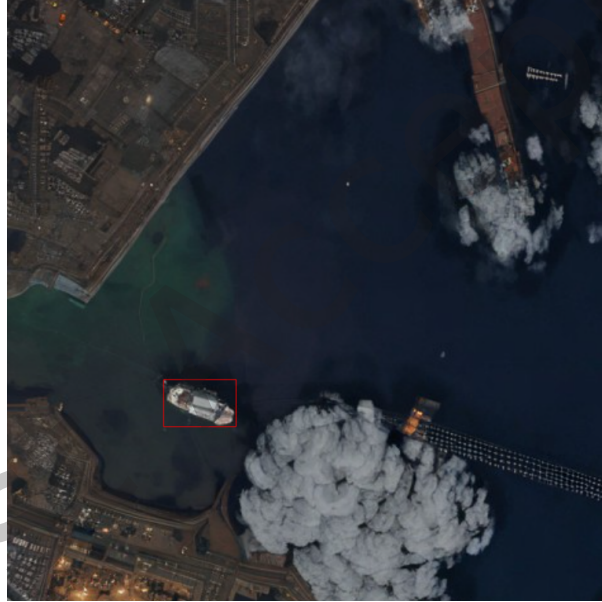


Fig. 5. Ship detected in an image generated with Stable Diffusion XL model

This capability is particularly powerful for testing Embodied AI, as it allows testers to simulate rare but critical edge cases (e.g., "a cargo ship partially obscured by fog near a coastline at night") that are logistically impossible to capture in the real world. The ability to probe for weaknesses on-demand in the perception system of the agent is a significant advantage over traditional testing.

Another way of generating new images using pre-trained models is using 3D models to 2D images. Among the available frameworks, two main types can be identified. On one hand, there are frameworks that come with preloaded 3D models of the environment (e.g., open sea, airports, inland) and objects (e.g., ships, planes, cars), allowing users to select from predefined options. Although these frameworks provide limited flexibility

in creating custom scenarios, they are powerful and efficient tools when their predefined solutions meet the application requirements. One such example is Rendered.ai [55], which generates images like the one shown in Figure 6. With Rendered.ai, you can select a range for the number of ships, specify the type of ships (such as cargo ships, fishing ships, etc.), and define the environment, including options for open sea or additional settings like ports. On the other hand, some frameworks allow the creation of a scenario from scratch, offering the flexibility to adapt the framework to a specific use case. However, this approach requires an additional effort to create the initial 3D models.



Fig. 6. Ships detected in an image generated with Rendered.ai

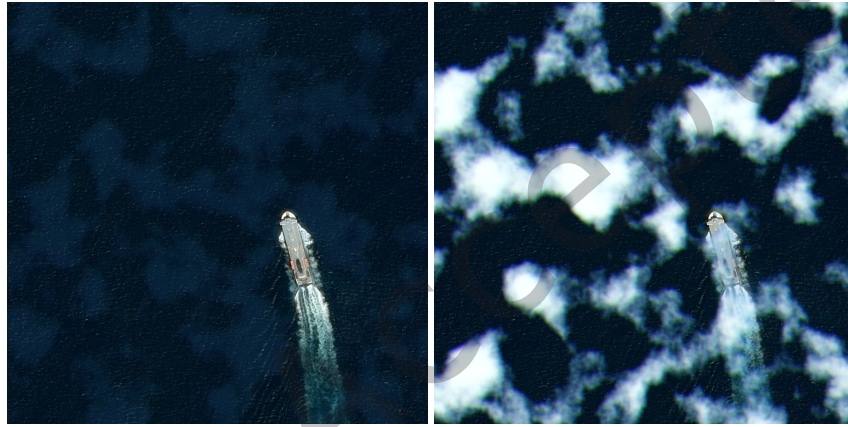
An important research direction in generative models is the automatic labelling of images, enabling their use not only for testing, but also for training. Several approaches have been proposed in the literature to achieve this, such as using stable diffusion models for background generation [34] and inpainting models for object placement [72], or combining 3D models with style transfer techniques such as CycleGAN to improve realism [12]. However, further research is needed in this area.

4.2 Modification of existing datasets

To show a different way to leverage synthetic data generation, this section evaluates the impact of augmenting the original dataset using the Satellite Cloud Generator tool. We first assess the performance of the initial model on the augmented dataset, then present and analyse the results of retraining the model with the synthetically enhanced data.

4.2.1 Generation of cloudy datasets and initial testing. The Satellite Cloud Generator tool utilises Perlin noise to synthetically add clouds to the existing dataset. Perlin noise is a widely used technique in AI, particularly for procedural generation, simulation, and data augmentation. The tool has several configurable parameters such as the minimum and maximum strength of the clouds, the locality degree, the channel misalignment or the cloud

colours. However, only two were modified from the default values. The first parameter is the clear threshold. By adjusting this parameter, the initial cloud can be truncated to prevent it from taking up excessive space in the original image. After testing different values for the clear threshold parameter, it was set to 0.4. With this value, a considerable number of clouds are added to the image, but the original image remains recognisable. The second parameter is the *max lvl* or maximum strength of the cloud. Adjusting this parameter between 0.0 and 1.0 controls the opacity of the cloud. A value of 0.0 indicates full transparency, while a value of 1.0 corresponds to full opacity. The difference between values 0.1 and 1.0 in this parameter can be seen in Figure 7. For the PoC, this parameter was set to range from 0.1 to 1.0 in increments of 0.1 so that the performance of the detection model could be evaluated at different levels of cloud opacity. It is worth noting that generating synthetic images using this technique is highly efficient compared to the time it would require to obtain similar images in the real world, with each image taking approximately 0.5 seconds to generate with the GPU specified earlier. Given that the training dataset contains 5000 images, the validation dataset has 1000 images, and the testing dataset includes other 1000 images, with a synthetic image generated for each opacity level for all of them, it took nearly 10 hours to generate the entire dataset for the PoC.



(a) Image generated with 0.1 cloud opacity (b) Image generated with 1.0 cloud opacity

Fig. 7. Satellite Cloud Generator generated images

Once the synthetic datasets (10 in total, one for each opacity value) were generated, the next step was to evaluate the performance of the initial detection model. As mentioned before, the metric used to evaluate the performance of the model is the mAP. This metric is widely used in machine learning and computer vision to evaluate the performance of the models, particularly for tasks such as object detection. It combines precision and recall, providing a single score or metric that reflects how well the model detects objects and assigns correct labels. To calculate the mAP, the mean of the Average-Precision (AP) scores is computed across all classes. However, in this case, since there is only one class, the mAP is equivalent to the AP. To calculate it, the bounding boxes predicted by the detection model and the ground truth are required. While it can be calculated manually, the process was simplified by using the `MeanAveragePrecision()` function from PyTorch. A minimum value of 0 indicates that the model is either not detecting anything or making highly inaccurate detections, while a maximum value of 1.0 (100%) represents perfect performance, with the model correctly detecting all objects. The results of this calculation are shown in Figure 8.

As mentioned earlier, the detection model was trained exclusively on images from the original training dataset (0.0 cloud opacity), where only a small percentage contained natural clouds. Consequently, the model achieves its

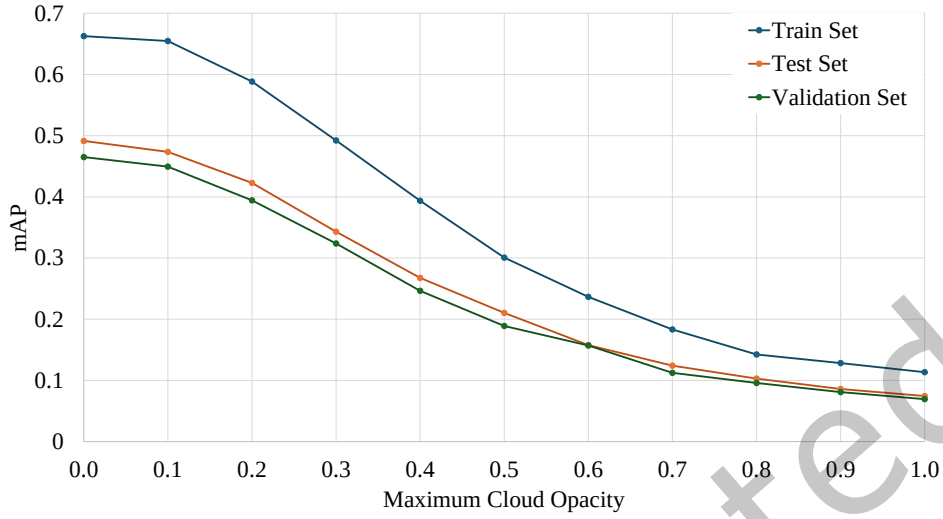


Fig. 8. Mean Average Precision calculated for all datasets and initial detection model

maximum performance across all sets for images without generated clouds. However, as cloud opacity increases, the model's performance begins to decline, dropping to about half its original effectiveness around a cloud opacity of 0.45 and performing poorly with dense clouds, showing an approximate 85% drop in performance at 1.0 cloud opacity for all three datasets. Furthermore, the metrics for the testing and validation datasets are nearly identical, while the training dataset consistently outperforms both across all opacity levels, as it is expected since it was the only set used during training. Figure 9 shows several examples where the original model fails to detect ships in images with different cloud opacities, despite successfully identifying them in cloud-free images.

The top-right image in Figure 9 is particularly concerning, as the cloud is very faint, yet the model fails to detect the ship due to the lack of training on similar images. This limitation could result in significantly poor performance in real-world deployment scenarios. Therefore, it is crucial at this point to train the model with different cloudy images during the retraining, enabling it to correctly detect ships in cloudy conditions.

4.2.2 Retraining the model with cloudy datasets. To evaluate the performance of the retrained model after training with synthetic datasets, five different models were trained. Specifically, datasets with opacity values of 0.1, 0.3, 0.5, 0.7 and 0.9 were used to obtain five different models, each starting from the original. The initial model training with the original dataset required approximately 200 epochs to converge, whereas the retraining process with each synthetic dataset was completed in around 50 epochs. The weights of the original model were not frozen during retraining, which means that there was a potential risk of the model failing to detect ships in cloud-free images.

The results of the retraining process for the training dataset are shown in Figure 10. These results visually confirm what was mentioned earlier: the model adapts only to the images and inputs it receives during the training. Since the weights of the original model were not frozen, it is expected that the model performs best with the dataset it was trained on. This is evident from the sharp performance peaks observed in each model when tested with the dataset used for retraining.

However, the results for the testing and validation datasets, shown in Figures 11 and 12 respectively, are more interesting. As mentioned earlier, by not freezing the weights during retraining, there was a potential risk that

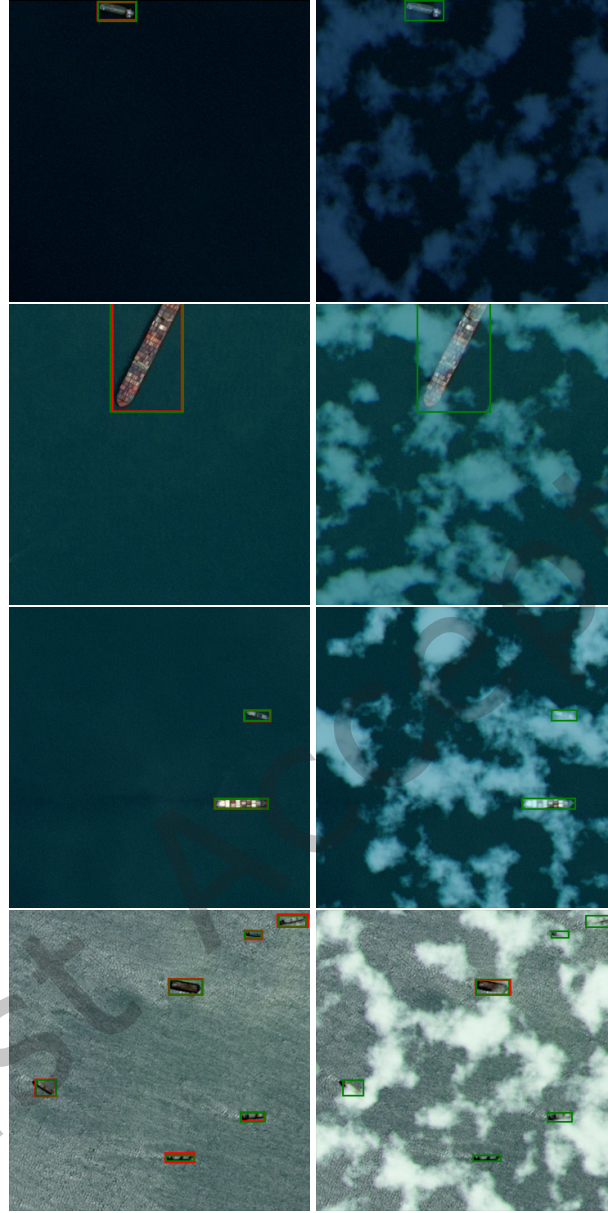


Fig. 9. Original model performance on original dataset (left) and synthetic datasets (right) with 0.3, 0.5, 0.7 and 0.9 cloud opacity, from top to bottom. Ground truth boxes in green. Prediction boxes in red.

the models might experience a performance drop on the cloud-free dataset. Indeed, there is a performance drop for the original dataset, but it is relatively small, as shown in Table 1, where the worst-case scenario is a 9.53% performance drop in the validation dataset for the model retrained with 0.9 cloud opacity images.

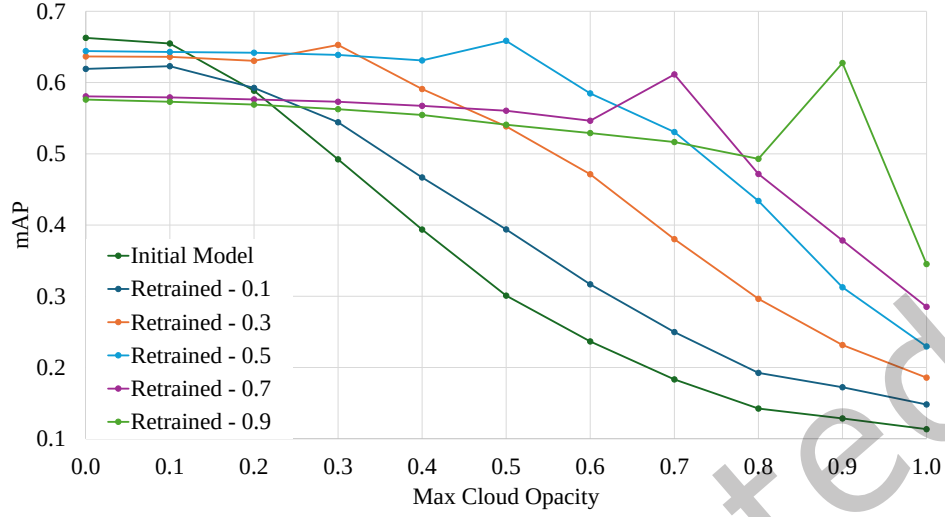


Fig. 10. Mean Average Precision calculated for the training set and all models

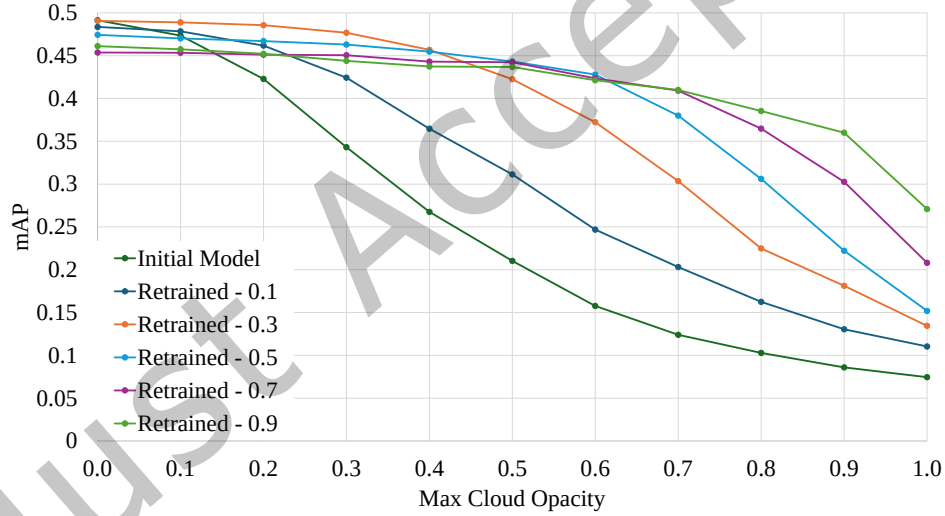


Fig. 11. Mean Average Precision calculated for the testing set and all models

For both the testing and validation datasets, the performance drop as cloud opacity increases remains evident, just as it was with the original model, especially for models re-trained with low cloud opacity (0.1 – 0.3). However, as cloud opacity increases in re-trained models, this performance drop becomes less pronounced, achieved by sacrificing performance at lower opacities. For example, as shown in Table 2, the model re-trained with 0.9 cloud opacity images performs worse than the original model up to the 0.2 opacity level, but the performance improvement across the remaining opacity levels is highly significant. With a performance drop of 6.17% and

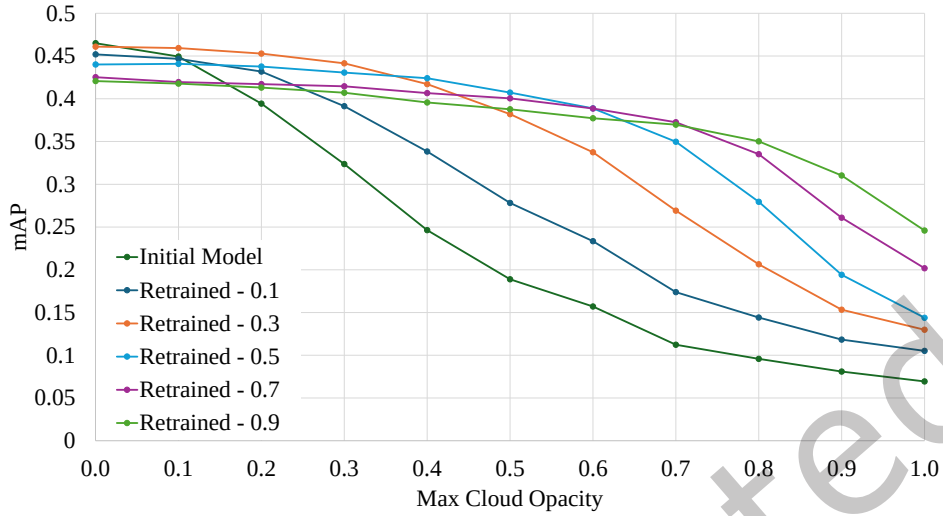


Fig. 12. Mean Average Precision calculated for the validation set and all models

Table 1. Relative performance drop for retrained models and cloud-free datasets

	Model 0.1	Model 0.3	Model 0.5	Model 0.7	Model 0.9
Test set	-1.59%	-0.16%	-3.48%	-7.68%	-6.17%
Validation set	-2.81%	-0.85%	-5.36%	-8.55%	-9.53%

3.40% on the original and 0.1 cloud opacity sets, respectively, the improvement at higher opacities reaches up to 318.77% at 0.9 cloud opacity.

To visualise the performance improvement, the same images were evaluated using the model trained with 0.9 cloud opacity images. The results are shown in Figure 13.

Now all ships are successfully detected even with clouds obscuring them. However, to truly validate the model's performance improvement, it must be demonstrated using real images in which clouds affect the ship surroundings or obscure them. In Figure 14, an image from the initial dataset depicts two ships, one partially affected by a cloud. The initial model successfully detects one of the ships, but fails to identify the one closer to the clouds. In contrast, the retrained model accurately detects both ships. On the other hand, in Figure 15, another image from the initial dataset shows one ship heavily obscured by clouds. The initial model fails to detect it, whereas the retrained model successfully identifies and annotates the ship, proving the model improvement not only on synthetic images but also on real ones.

In conclusion, the utility of generative AI in testing is successfully demonstrated. Through the use of synthetic datasets with varying levels of cloud opacity, it was shown that the performance of the detection model could be effectively tested and, more importantly, improved, enhancing its robustness in cloudy scenarios while approximately preserving its effectiveness in cloud-free conditions. However, the individual models tested still showed performance limitations. Although the re-trained models performed well in cloudier scenarios, the overall detection accuracy across all opacity levels could be further improved.

Table 2. Mean average precision performance change for the testing dataset and retrained models

	Maximum Cloud Opacity										
	0.0	0.1	0.2	0.3	0.4	0.5	0.6	0.7	0.8	0.9	1.0
Initial Model	0.491	0.747	0.423	0.343	0.268	0.210	0.158	0.124	0.103	0.086	0.074
Model 0.1	0.484	0.478	0.462	0.424	0.365	0.311	0.247	0.203	0.163	0.130	0.110
Abs diff	-0.008	0.005	0.039	0.081	0.097	0.101	0.089	0.079	0.060	0.044	0.036
Rel diff (%)	-1.59	1.04	9.20	23.65	36.31	48.00	56.56	63.69	57.99	51.66	48.18
Model 0.3	0.491	0.489	0.486	0.477	0.457	0.423	0.372	0.304	0.225	0.181	0.134
Abs diff	-0.001	0.015	0.063	0.134	0.189	0.212	0.215	0.179	0.122	0.095	0.060
Rel diff (%)	-0.16	3.24	14.83	38.97	70.80	100.84	136.03	144.69	118.49	110.80	80.47
Model 0.5	0.474	0.470	0.467	0.463	0.455	0.443	0.428	0.380	0.306	0.222	0.152
Abs diff	-0.017	-0.003	0.044	0.120	0.187	0.233	0.270	0.256	0.203	0.136	0.077
Rel diff (%)	-3.48	-0.70	10.46	34.93	70.00	110.70	171.27	206.31	197.46	158.43	103.92
Model 0.7	0.454	0.453	0.451	0.450	0.443	0.442	0.424	0.409	0.365	0.303	0.208
Abs diff	-0.038	-0.020	0.028	0.107	0.176	0.232	0.266	0.285	0.262	0.217	0.134
Rel diff (%)	-7.68	-4.25	6.69	31.30	65.63	110.29	168.52	229.87	254.57	252.03	179.24
Model 0.9	0.461	0.457	0.452	0.444	0.437	0.437	0.421	0.410	0.385	0.360	0.271
Abs diff	-0.030	-0.016	0.029	0.101	0.170	0.226	0.264	0.286	0.283	0.274	0.196
Rel diff (%)	-6.17	-3.40	6.95	29.42	63.50	107.63	167.08	230.42	274.57	318.77	263.53

While the current PoC demonstrates promising results regarding the utility of generative AI in software testing, further improvements could be explored in future work. One possible direction is the construction of a larger and more diverse training dataset that combines original images with synthetic images over a wider range of cloud opacity levels. This could help improve detection performance more uniformly across all environmental conditions. Another potential line of investigation is the integration of multiple models, such as using a dedicated cloud detection module as a preprocessing stage to inform the selection or adaptation of the ship detection model. These ideas, while beyond the scope of the current study, represent logical extensions that could build upon the demonstrated success of our approach and further enhance its applicability in satellite systems.

5 Conclusions

This work has examined the role of AI in the domain of software testing, with particular emphasis on applications to Embodied AI systems. We provided a structured taxonomy of existing AI techniques for testing complex systems and analysed how these methods complement conventional approaches. The main contribution of this study is a PoC focusing on a satellite as an embodied agent. We demonstrated that validating the perception system of such an agent is a central challenge, given the dynamic and unpredictable nature of its operating environment. Using generative AI to synthetically produce datasets with varying levels of cloud opacity, we systematically tested the robustness of a ship detection model.

Our evaluation produced two key findings. First, we showed that while the baseline model performed well in clear conditions, its accuracy degraded significantly under cloud cover, underscoring the risks of relying on limited training data. Second, we demonstrated that retraining with synthetically generated data improved robustness under adverse conditions, with benefits confirmed on both synthetic and real-world images. These

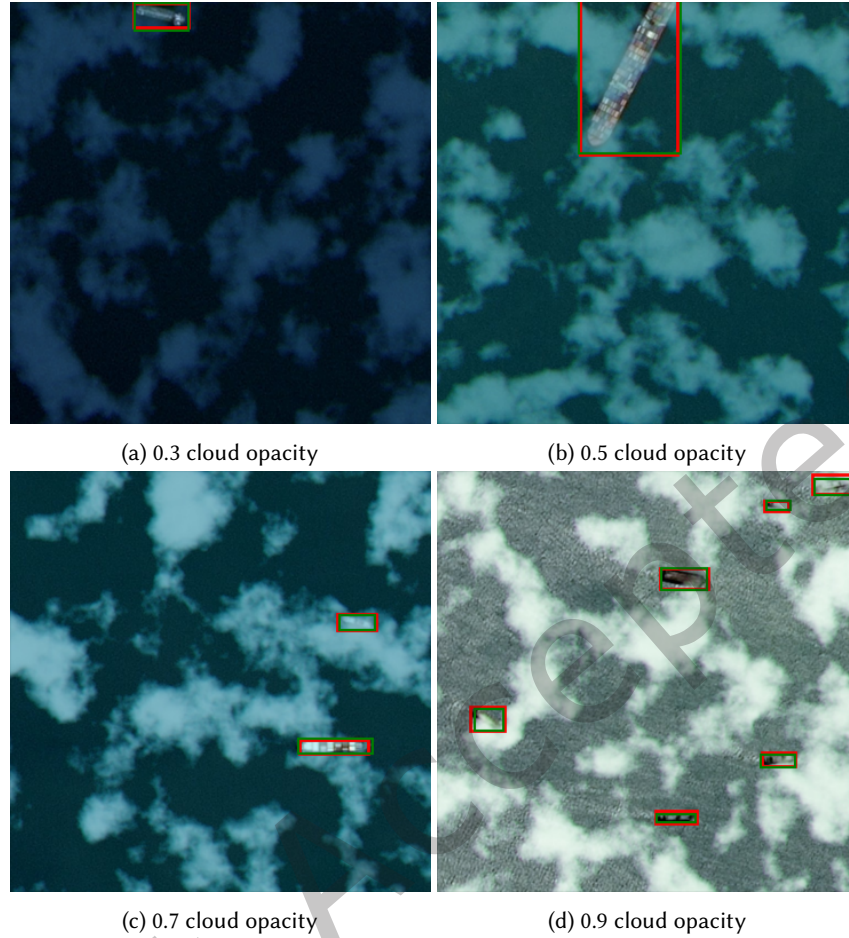


Fig. 13. Retrained model (0.9) performance on synthetic dataset. Ground truth boxes in green. Prediction boxes in red

results indicate that AI-driven testing can diversify training data and strengthen validation processes while reducing reliance on costly real-world data collection.

At the same time, this work highlighted several open challenges. These include the high computational cost of training generative models and the need for more diverse datasets. Future research could explore the integration of dedicated cloud detection models as a pre-processing step to optimise performance. In addition, hybrid methodologies that combine simulation with deep learning offer potential for mitigating computational overhead while maintaining model accuracy. Addressing these challenges will be important for advancing the validation of embodied AI systems.

Acknowledgments

The research presented throughout this paper has received funding from the European Commission's Horizon Europe programme under the METASAT project (grant agreement 101082622) and from the Basque Government through the CERVERA programme within the MEDUSA project (grant number CER-20231011).

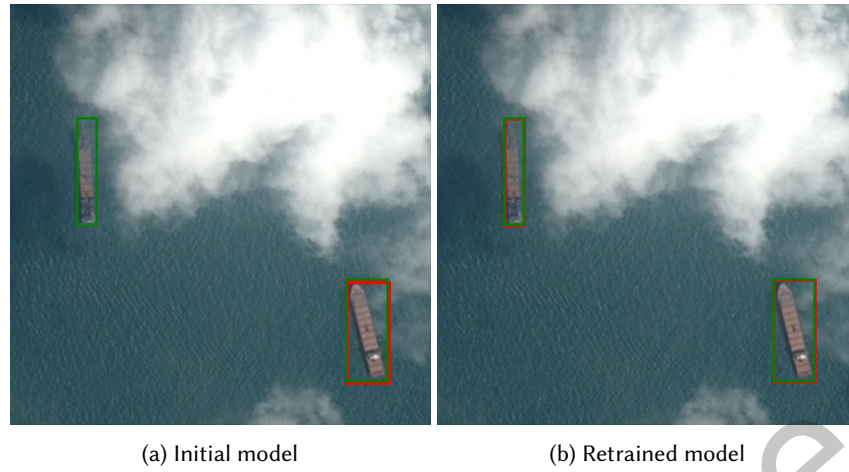


Fig. 14. Model performance on real image with clouds and two ships. Ground truth boxes in green. Prediction boxes in red

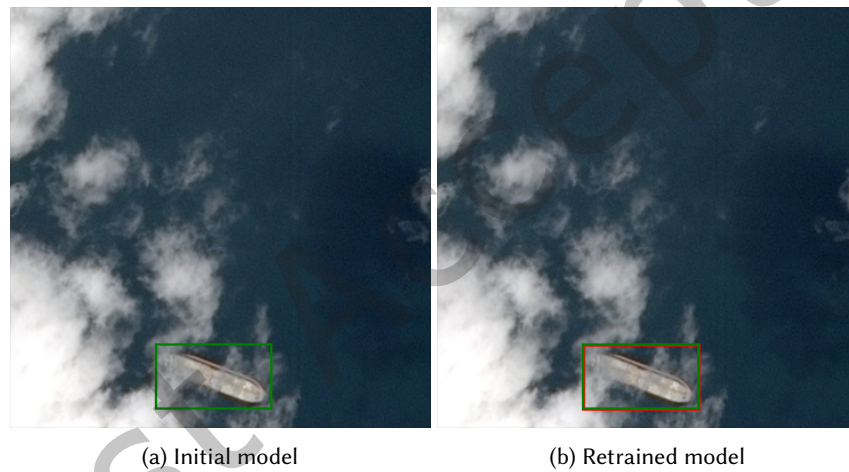


Fig. 15. Model performance on real image with clouds and one ship. Ground truth boxes in green. Prediction boxes in red

References

- [1] Mohamed Abdelkarim and Reem ElAdawi. 2022. TCP-Net: Test Case Prioritization using End-to-End Deep Neural Networks. In *International Conference on Software Testing, Verification and Validation Workshops (ICSTW)*. 122–129.
- [2] Ahmed Abdu, Zhengjun Zhai, Hakim A Abdo, and Redhwan Algabri. 2024. Software Defect Prediction based on Deep Representation Learning of Source Code From Contextual Syntax and Semantic Graph. *IEEE Transactions on Reliability* (2024).
- [3] Airbus DS Intelligence. 2018. Airbus Ship Detection Challenge. <https://www.kaggle.com/c/airbus-ship-detection/overview>. Kaggle. Accessed January 2025.
- [4] Ohood Al-Masri and Wedad Abdulraqueeb Al-Sorori. 2022. Object-Oriented Test Case Generation Using Teaching Learning-Based Optimization (TLBO) Algorithm. *IEEE Access* 10 (2022), 110879–110888.
- [5] Misbah Ali, Tehseen Mazhar, Yasir Arif, Shaha Al-Otaibi, Yazeed Yasin Ghadi, Tariq Shahzad, Muhammad Amir Khan, and Habib Hamam. 2024. Software defect prediction using an intelligent ensemble-based model. *IEEE Access* (2024).

- [6] Umair Ali, Shabib Aftab, Ahmed Iqbal, and Zahid Nawaz. 2020. Software Defect Prediction using Variant based Ensemble Learning and Feature Selection Techniques. *International Journal of Modern Education & Computer Science* 12, 5 (2020), 29–40.
- [7] Domenico Amalfitano, Stefano Faralli, Jean Carlo Rossa Hauck, Santiago Matalonga, and Damiano Distante. 2023. Artificial intelligence applied to software testing: A tertiary study. *Comput. Surveys* 56, 3 (2023), 1–38.
- [8] Mohammad Amimul Ihsan Aquil and Wan Hussain Wan Ishak. 2020. Predicting Software Defects using Machine Learning Techniques. *International Journal of Advanced Trends in Computer Science and Engineering* 9, 4 (2020), 6609–6616.
- [9] Bahman Arasteh, Mohammad Reza Sattari, and Reza Shokri Kalan. 2022. Fuzuli: Automatic Test Data Generation for Software Structural Testing using Grey Wolf Optimization Algorithm and Genetic Algorithm. In *Intl Conf on Dependable, Autonomic and Secure Computing, Intl Conf on Pervasive Intelligence and Computing, Intl Conf on Cloud and Big Data Computing, Intl Conf on Cyber Science and Technology Congress (DASC/PiCom/CBDCCom/CyberSciTech)*. 1–6.
- [10] Marwa Assim, Qasem Obeidat, and Mustafa Hammad. 2020. Software Defects Prediction using Machine Learning Algorithms. In *International conference on data analytics for business and industry: way towards a sustainable economy (ICDABI)*. 1–6.
- [11] Mojtaba Bagherzadeh, Nafiseh Kahani, and Lionel Briand. 2021. Reinforcement Learning for Test Case Prioritization. *IEEE Transactions on Software Engineering* 48, 8 (2021), 2836–2856.
- [12] Ruud Barth, Jochen Hemming, and Eldert J Van Henten. 2020. Optimising Realism of Synthetic Images using Cycle Generative Adversarial Networks for Improved Part Segmentation. *Computers and Electronics in Agriculture* 173 (2020), 105378.
- [13] Márcio P. Basgalupp, Rodrigo C. Barros, Tiago S. Da Silva, Fábio F. Silveira, Pérciles B. C. Miranda, and Ferrante Neri. 2024. An Experimental Analysis on Automated Machine Learning for Software Defect Prediction. In *Congress on Evolutionary Computation (CEC)*. 1–8.
- [14] Emin Borandag. 2023. Software Fault Prediction using an RNN-based Deep Learning Approach and Ensemble Machine Learning Techniques. *Applied Sciences* 13, 3 (2023), 1639.
- [15] Jinkang Cai, Weiwen Deng, Haoran Guang, Ying Wang, Jiangkun Li, and Juan Ding. 2022. A survey on data-driven scenario generation for automated vehicle testing. *Machines* 10, 11 (2022), 1101.
- [16] Fanliang Chen, Zheng Li, Ying Shang, and Yang Yang. 2022. Focus on New Test Cases in Continuous Integration Testing based on Reinforcement Learning. In *2022 IEEE 22nd International Conference on Software Quality, Reliability and Security (QRS)*. 830–841.
- [17] Mikolaj Czerkawski, Robert Atkinson, Craig Michie, and Christos Tachtatzis. 2023. SatelliteCloudGenerator: Controllable Cloud and Shadow Synthesis for Multi-Spectral Optical Satellite Images. *Remote Sensing* 15, 17 (2023).
- [18] Enrique A. Da Roza, Jackson A. Prado Lima, Rogério C. Silva, and Silvia Regina Vergilio. 2022. Machine Learning Regression Techniques for Test Case Prioritization in Continuous Integration Environment. In *International Conference on Software Analysis, Evolution and Reengineering (SANER)*. 196–206.
- [19] Christine Dewi, Rung-Ching Chen, Yan-Ting Liu, and Shao-Kuo Tai. 2022. Synthetic Data Generation using DCGAN for Improved Traffic Sign Recognition. *Neural Computing and Applications* 34, 24 (2022), 21465–21480.
- [20] Radu Găceanu, Arnold Szederjesi-Dragomir, and Andreea Vescan. 2024. Leveraging Rough Sets for Enhanced Test Case Prioritization in a Continuous Integration Context. In *International Conference on Software Analysis, Evolution and Reengineering-Companion (SANER-C)*. 175–182.
- [21] Luo Gang. 2023. Genetic Algorithm and its Application in Software Test Data Generation. In *International Conference on Applied Intelligence and Sustainable Computing (ICAISC)*. 1–6.
- [22] Zheng Ge, Songtao Liu, Feng Wang, Zeming Li, and Jian Sun. 2021. YOLOX: Exceeding YOLO Series in 2021. *arXiv preprint arXiv:2107.08430* (2021).
- [23] Ian Goodfellow, Jean Pouget-Abadie, Mehdi Mirza, Bing Xu, David Warde-Farley, Sherjil Ozair, Aaron Courville, and Yoshua Bengio. 2014. Generative Adversarial Nets. In *Advances in Neural Information Processing Systems*, Z. Ghahramani, M. Welling, C. Cortes, N. Lawrence, and K.Q. Weinberger (Eds.), Vol. 27. Curran Associates, Inc. https://proceedings.neurips.cc/paper_files/paper/2014/file/5ca3e9b122f61f8f06494c97b1afccf3-Paper.pdf
- [24] Xiujiang Guo. 2021. Towards Automated Software Testing with Generative Adversarial Networks. In *International Conference on Dependable Systems and Networks-Supplemental Volume (DSN-S)*. 21–22.
- [25] Xiujiang Guo, Hiroyuki Okamura, and Tadashi Dohi. 2022. Automated Software Test Data Generation with Generative Adversarial Networks. *IEEE Access* 10 (2022), 20690–20700.
- [26] Rajendra Gurram, M Babu Reddy, Bindu Madhavi Tummala, Kanakamedala Phaneendra, Pamulapati Ashok Reddy, and D Durga Prasad. 2024. A Novel Ant Colony based Genetic Approach Neural Learning for Defect Prediction in Software Products. In *International Conference on Emerging Systems and Intelligent Computing (ESIC)*. 330–337.
- [27] Hussam Hourani, Ahmad Hammad, and Mohammad Lafi. 2019. The impact of artificial intelligence on software testing. In *2019 IEEE Jordan International Joint Conference on Electrical Engineering and Information Technology (JEEIT)*. IEEE, 565–570.
- [28] Luosen Huang, Hengyuan Liu, Yong Liu, Ying Shang, and Zheng Li. 2024. A Generative Adversarial Imitation Learning Method for Continuous Integration Testing. In *International Seminar on Artificial Intelligence, Networking and Information Technology (AINIT)*. 1084–1089.

- [29] Yechao Huang, Ting Shu, and Zuohua Ding. 2021. A Learn-to-Rank Method for Model-Based Regression Test Case Prioritization. *IEEE Access* 9 (2021), 16365–16382.
- [30] Tero Karras, Samuli Laine, and Timo Aila. 2021. A Style-Based Generator Architecture for Generative Adversarial Networks. *IEEE Transactions on Pattern Analysis and Machine Intelligence* 43, 12 (2021), 4217–4228. doi:10.1109/TPAMI.2020.2970919
- [31] Amandeep Kaur. 2020. An Approach to Extract Optimal Test Cases using AI. In *International Conference on Cloud Computing, Data Science & Engineering (Confluence)*. IEEE, 649–654.
- [32] Abdulrahman Ahmed Khudhur, Ibrahim Mohamed, and Suhaila Zainudin. 2024. Enhancing Automated Test Case Generation through KNN Algorithm in Software Testing. *Journal of Electrical Systems* 20, 3s (2024), 927–936.
- [33] Leonidas Kosmidis, Alejandro J. Calderón, Aridane Álvarez, Stefano Sinisi, Eckart Göhler, Paco Gómez, Alfred Hönl, Álvaro Jover, Lorenzo Lazzara, Miguel Masmano, Peio Onaindia, Tomaso Poggi, Iván Rodríguez, Marc Solé, Giulia Stazi, Martina Maria Trompouki, Alessandro Ulisse, Valerio Di Valerio, Jannis Wolf, and Irune Yarza. 2023. METASAT: Modular Model-Based Design and Testing for Applications in Satellites. In *Embedded Computer Systems: Architectures, Modeling, and Simulation*. Springer Nature Switzerland, 347–362.
- [34] Yuhang Li, Xin Dong, Chen Chen, Weiming Zhuang, and Lingjuan Lyu. 2024. A Simple Background Augmentation Method for Object Detection with Diffusion Model. In *European Conference on Computer Vision*. Springer, 462–479.
- [35] Zhenpeng Liu, Xianwei Yang, Shichen Zhang, Yi Liu, Yonggang Zhao, and Weihua Zheng. 2022. Automatic Generation of Test Cases based on Genetic Algorithm and RBF Neural Network. *Mobile Information Systems* 2022, 1 (2022), 1489063.
- [36] Nan Luo, Ying Ma, Jianmin Li, and Yanqi Xie. 2023. A Just-in-time Software Defect Detection Method using Generative Adversarial Networks. In *International Conference on Electronic Communication and Artificial Intelligence (ICECAI)*. 37–45.
- [37] Alok Mathur, Shreyaan Pradhan, Prasoon Soni, Dhruvil Patel, and Rajeshkannan Regunathan. 2023. Automated Test Case Generation using T5 and GPT-3. In *International Conference on Advanced Computing and Communication Systems (ICACCS)*, Vol. 1. 1986–1992.
- [38] Iqra Mehmood, Sidra Shahid, Hameed Hussain, Inayat Khan, Shafiq Ahmad, Shahid Rahman, Najeeb Ullah, and Shamsul Huda. 2023. A Novel Approach to Improve Software Defect Prediction Accuracy using Machine Learning. *IEEE Access* 11 (2023), 63579–63597.
- [39] Hajar Mirzaei and Mohammad Reza Keyvanpour. 2022. Reinforcement Learning Reward Function for Test Case Prioritization in Continuous Integration. In *Iranian Joint Congress on Fuzzy and Intelligent Systems (CFIS)*. 1–6.
- [40] Brian B Moser, Arundhati S Shanbhag, Federico Raue, Stanislav Frolov, Sebastian Palacio, and Andreas Dengel. 2024. Diffusion Models, Image Super-Resolution, and Everything: A Survey. *IEEE Transactions on Neural Networks and Learning Systems* (2024).
- [41] Sara Moshdari, Joanna C. S. Santos, Mehdi Mirakhorli, and Ahmet Okutan. 2020. Looking for Software Defects? First Find the Nonconformists. In *International Working Conference on Source Code Analysis and Manipulation (SCAM)*. 75–86.
- [42] Lahbib Naimi, Mohamed Manaouch, Abdeslam Jakim, et al. 2024. A New Approach for Automatic Test Case Generation from Use Case Diagram using LLMs and Prompt Engineering. In *International Conference on Circuit, Systems and Communication (ICCS)*. 1–5.
- [43] Mehwish Naz, Zaeem Anwaar, and Wasi Haider Butt. 2023. Automated White Box Test Case Generation for Statement Coverage using U-NSGA-III. In *International Conference on Open Source Systems and Technologies (ICOSST)*. 1–6.
- [44] Jiaojiao Nie, Xianglin Fan, and Yushuang Wu. 2021. Test Case Generation Method based on Generative Adversarial Network. In *Journal of Physics: Conference Series*, Vol. 1883.
- [45] RJT Nirmalraj, Pon Ramalingam, V Sreetharan, PMD Ali Khan, N Divya, and Muruganantham Ponnusamy. 2023. An End-to-End S-AF-RCNN based Framework for Just-in-Time Defect Prediction. In *International Conference on Innovative Mechanisms for Industry Applications (ICIMIA)*. 305–310.
- [46] NVIDIA Corporation. 2025. Stable Diffusion XL. <https://build.nvidia.com/stabilityai/stable-diffusion-xl>. Accessed January 2025.
- [47] Mitchell Olsthoorn. 2022. More Effective Test Case Generation with Multiple Tribes of AI. In *Proceedings of the ACM/IEEE 44th International Conference on Software Engineering: Companion Proceedings*. 286–290.
- [48] Safa Omri and Carsten Sinz. 2022. Learning to Rank for Test Case Prioritization. In *Proceedings of the 15th Workshop on Search-Based Software Testing*. 16–24.
- [49] Prasanta Panda, Debaryaan Sahoo, and Debarjun Sahoo. 2024. Automating Fault Prediction in Software Testing using Machine Learning Techniques: A Real-World Applications. In *International Conference on Sustainable Computing and Smart Systems (ICSSS)*. 841–844.
- [50] Zhongyuan Qin, Jiarong Fan, Zeru Li, Xujian Liu, and Xin Sun. 2023. CWGAN-GP: Fuzzing Testcase Generation Method based on Conditional Generative Adversarial Network. In *International Conference on Trust, Security and Privacy in Computing and Communications (TrustCom)*. 1286–1293.
- [51] Xiongwei Qiu, Pengtong Fan, and Jiale Ren. 2023. Convolutional Neural Network-based Research on Software Engineering Defect Prediction. In *Proceedings of the 6th International Conference on Information Technologies and Electrical Engineering*. 305–308.
- [52] Alec Radford, Luke Metz, and Soumith Chintala. 2016. Unsupervised Representation Learning with Deep Convolutional Generative Adversarial Networks. arXiv:1511.06434 <https://arxiv.org/abs/1511.06434>
- [53] Manikandan Rajagopal, Ramkumar Sivasakthivel, Karuppusamy Loganathan, and Ioannis E Sarris. 2023. An Automated Path-Focused Test Case Generation with Dynamic Parameterization using Adaptive Genetic Algorithm (AGA) for Structural Program Testing. *Information* 14, 3 (2023), 166.

- [54] Seema Rani and Amandeep Kaur. 2020. Efficient Framework for Fully Automatic Test Case Generation and Prioritization using Genetic Algorithm in Software Testing. *Journal of Computational and Theoretical Nanoscience* 17, 11 (2020), 5198–5204.
- [55] Rendered.ai. 2025. Unlimited Synthetic Data Generation For Computer Vision. <https://rendered.ai/>. Accessed January 2025.
- [56] S. M. Rifat, Akil Uddin Bhuyain, Md. Shamim Hossain, Md. Solaiman Mia, and Mahmuda Rahman. 2023. A Systematic Approach for Enhancing Software Defect Prediction using Machine Learning. In *International Conference on Next-Generation Computing, IoT and Machine Learning (NCIM)*.
- [57] Lukas Rosenbauer, Anthony Stein, Roland Maier, David Pätz, and Hähner Jörg. 2020. XCS as a Reinforcement Learning Approach to Automatic Test Case Prioritization. In *GECCO '20: Proceedings of the 2020 Genetic and Evolutionary Computation Conference Companion*. 1798–1806.
- [58] Priyanka Sasikumar and Subhashni Raj. 2021. Automatic Test Case Generation Using Hybrid Genetic Algorithm. In *International Conference on Electronics and Sustainable Communication Systems (ICESC)*. 1549–1556.
- [59] Priyanka Dattatray Sawant. 2024. Test Case Prioritization for Regression Testing Using Machine Learning. In *International Conference on Artificial Intelligence Testing (AITest)*. 152–153.
- [60] Tri Agus Setiawan, Haryono Soeparno, Ford Lumban Gaol, and Yulyani Arifin. 2023. Neural Network and Resampling to Handle Imbalance Class on Estimate Software Defect. In *International Congress on Advanced Applied Informatics Winter (IIAI-AAI-Winter)*. 296–299.
- [61] Sahana P Shankar, Divyansh Pushp, Nemi Makadia, Raghav Dubey, Anusha Nayak, and Shilpa Shashikant Chaudhari. 2023. Software Defect Predictor and Classifier Tool using Machine Learning Techniques. In *International Conference on Network, Multimedia and Information Technology (NMITCON)*. 1–6.
- [62] Ulligaddala Srinivasarao, Akhil Chaganti, Ajay Mannam, Dhanush Kotrikeline, and Hemanth Katta. 2024. Software Bug Report Detection Methods based on Machine Learning Techniques. In *International Conference on Advancement in Computation & Computer Technologies (InCACCT)*. 99–104.
- [63] David Steenari, Leonidas Kosmidis, Ivan Rodríguez-Ferrández, Alvaro Jover-Alvarez, and Kyra Förster. 2021. OBPMark (On-Board Processing Benchmarks) - Open Source Computational Performance Benchmarks for Space Applications. In *European Workshop on On-Board Data Processing (OBDP)*. 14–17.
- [64] David Steenari, Leonidas Kosmidis, Ivan Rodríguez-Ferrández, Alvaro Jover-Alvarez, and Kyra Förster. 2021. On-Board Processing Benchmarks. <http://obpmark.github.io/>.
- [65] Xin Sun, Wen Wang, Xujian Liu, Jiarong Fan, Zeru Li, Yubo Song, and Zhongyuan Qin. 2022. VecSeeds: Generate Fuzzing Testcases from Latent Vectors based on VAE-GAN. In *International Conference on Trust, Security and Privacy in Computing and Communications (TrustCom)*. IEEE, 953–958.
- [66] Kung Tang, Yutang Zhang, and Shengkai Xia. 2024. Software Defect Prediction and Automatic Repair based on Machine Learning. *Highlights in Science, Engineering and Technology* 98 (2024), 439–443.
- [67] Michele Tufano, Dawn Drain, Alexey Svyatkovskiy, and Neel Sundaresan. 2022. Generating Accurate Assert Statements for Unit Test Cases using Pretrained Transformers. In *Proceedings of the 3rd ACM/IEEE International Conference on Automation of Software Test*. 54–64.
- [68] Muhammad Waqar, Imran, Muhammad Atif Zaman, Muhammad Muzammal, and Jungsuk Kim. 2022. Test Suite Prioritization based on Optimization Approach using Reinforcement Learning. *Applied Sciences* 12, 13 (2022), 6772.
- [69] Abebaw Worku, Esubalew Alemneh, Yibeltal Assefa, Tadilo Getaneh, and Siraye Mekonnen. 2023. Test Case Generation from Quality Attribute Scenarios using Machine Learning Approach. In *International Conference on Information and Communication Technology for Development for Africa (ICT4DA)*. 31–36.
- [70] Ahmadreza Saboor Yaraghi, Mojtaba Bagherzadeh, Nafiseh Kahani, and Lionel C Briand. 2022. Scalable and accurate test case prioritization in continuous integration contexts. *IEEE Transactions on Software Engineering* 49, 4 (2022), 1615–1639.
- [71] Muhammad Nouman Zafar, Wasif Afzal, Eduard Paul Enoiu, Zulqarnain Haider, and Inderjeet Singh. 2024. Optimizing Model-based Generated Tests: Leveraging Machine Learning for Test Reduction. In *International Conference on Software Testing, Verification and Validation Workshops (ICSTW)*. 44–54.
- [72] Lingzhi Zhang, Tarmily Wen, Jie Min, Jiancong Wang, David Han, and Jianbo Shi. 2020. Learning Object Placement by Inpainting for Compositional Data Augmentation. In *Computer Vision—ECCV 2020: 16th European Conference, Glasgow, UK, 2020, Proceedings, Part XIII* 16. Springer, 566–581.
- [73] Yifan Zhao, Dan Hao, and Lu Zhang. 2023. Revisiting Machine Learning based Test Case Prioritization for Continuous Integration. In *International Conference on Software Maintenance and Evolution (ICSME)*. 232–244.

Received 1 February 2025; revised 5 September 2025; accepted 23 November 2025